南　　开　　大　　学

密 码 与 网 络 空 间 安 全 学 院

网络安全技术

Lab1：基于 DES 加密的网络聊天程序

肖俊涛

年级：2023 级

专业：信息安全与法学双学位班

指导教师：张玉

2026 年 4 月 18 日

目录

一、 实验要求 **1**

二、 实验环境 **1**

（一） 软硬件环境 1

（二） 工程结构 1

（三） 编译与运行方式 1

（四） 命令行操作与参数说明 2

三、 实验原理 **4**

（一） DES 加密算法 4

（二） TCP 协议与可靠传输机制 5

（三） Socket 编程与程序工作流程 7

（四） 预共享密钥与本实验的完整通信过程 8

四、 实验核心代码解析 **9**

五、 实验结果和分析 **12**

六、 收获与反思 **14**

一、 实验要求

本实验的目标是完成一个**基于 DES 对称加密的 TCP 聊天程序**，结合课程要求与本次实验，实验要求如下。

1. 理解 DES 分组加密算法原理，并独立完成字符串消息的加密与解密。
2. 理解 Socket 编程流程，完成基于 TCP 的一对一聊天程序。
3. 将 DES 与 TCP 通信结合，实现**发送方加密**、**接收方解密**的保密通信过程。

二、 实验环境

(一) 软硬件环境

项目	说明
宿主系统	Windows 平台
实验运行环境	WSL2 下的 Ubuntu Linux
编译器	`g++`（支持 C++17）
构建工具	`CMake`（工程文件中最低版本要求为 3.16）
主要语言	C++17
网络编程接口	Linux Socket API
线程支持	`std::thread`、`std::atomic`

(二) 工程结构

本实验采用多文件方式组织代码，目录结构清晰，便于分别实现加密算法和聊天功能。

```
Lab01
    CMakeLists.txt
    README.md
    main.cpp
    include
        DES_Operation.h
        chat.h
    src
        DES_Operation.cpp
        chat.cpp
```

(三) 编译与运行方式

本实验最终采用 CMake 构建工程，核心命令如下：

```
cmake -S . -B build
cmake --build build
```

编译后可执行文件输出到 `bin` 目录中。程序分别以服务器端和客户端模式运行：

```
./bin/DES_chat server --port 8888 --key SecLab01
./bin/DES_chat client --ip 127.0.0.1 --port 8888 --key SecLab01
```

其中，双方需事先约定相同的 8 字节 DES 密钥，并通过 `--key` 参数传入。

（四） 命令行操作与参数说明

由于本实验程序运行在 Linux 命令行环境下，因此有必要对实际操作命令及其参数含义进行统一说明。实验中使用的典型命令序列如下。

(1) 进入 Linux 环境并编译工程

```
>wsl -d Ubuntu-OSLab-Recovered
cd /mnt/c/Users/xjt26/Desktop/网络安全技术
sudo apt update
sudo apt install -y cmake g++
cmake -S . -B build
cmake --build build
```

上述命令的作用依次为：进入 WSL 中的 Ubuntu 发行版、切换到实验工程目录、安装构建工具与编译器、生成构建目录并完成 CMake 编译。编译完成后，可执行文件输出到 `bin` 目录中。

(2) 服务端与客户端启动命令

本实验需要打开两个终端，分别启动服务端与客户端。服务端命令如下：

```
./bin/DES_chat server --port 8888 --key SecLab01
```

客户端命令如下：

```
./bin/DES_chat client --ip 127.0.0.1 --port 8888 --key SecLab01
```

若希望在聊天过程中显示十六进制密文，则可以额外加入 `--show-cipher yes`，例如：

```
./bin/DES_chat client --ip 127.0.0.1 --port 8888 --key SecLab01 --show-cipher
    yes
```

若命令中省略 `--show-cipher`，程序会在建立聊天前通过命令行询问用户是否显示密文。

(3) 运行参数含义

程序运行时涉及的主要参数如表 1 所示。

表 1: 程序运行命令及参数含义

参数或命令	含义说明
`server`	指定程序以服务器端模式运行，负责监听端口并等待客户端连接
`client`	指定程序以客户端模式运行，负责主动连接服务器
`--ip 127.0.0.1`	指定客户端连接的目标 IP 地址；`127.0.0.1` 表示本机回环地址，适用于在同一台主机上进行测试
`--port 8888`	指定通信端口号；服务端监听该端口，客户端也必须连接同一端口
`--key SecLab01`	指定双方预先约定的 8 字节 DES 密钥；两端必须保持一致，否则无法正确解密
`--show-cipher yes/no`	指定是否在终端中显示十六进制密文；用于观察加密结果和展示实验效果
`quit`	本地退出控制命令；输入后程序直接关闭本端连接，而不会将其作为普通聊天消息发送

(4) 命令行交互在本实验中的作用

在本实验中，命令行参数不仅用于区分程序身份，还承担了配置实验环境的重要功能。通过 `server/client` 参数可以在同一份可执行程序中切换不同角色；通过 `--ip` 和 `--port` 参数可以指定连接目标；通过 `--key` 参数实现预共享密钥输入；通过 `--show-cipher` 参数则可以决定是否在运行过程中显示密文。这种设计使程序的实际使用方式清晰准确。

三、 实验原理

(一) DES 加密算法

DES（Data Encryption Standard）是一种典型的**对称分组密码**。其明文分组长度为 64 bit，密钥长度为 64 bit，其中 8 bit 用作奇偶校验，因此有效密钥长度为 56 bit。DES 的整体结构属于 **Feistel 网络**，优点是加密和解密可以复用同一轮函数，仅需将子密钥的使用顺序反转。

设 64 bit 明文分组记为

$$M \in \{0,1\}^{64}, \tag{1}$$

初始密钥记为

$$K \in \{0,1\}^{64}. \tag{2}$$

首先对明文执行初始置换 `IP`，得到

$$IP(M) = L_0 \| R_0, \tag{3}$$

其中 $L_0, R_0 \in \{0,1\}^{32}$，分别表示左右两个 32 bit 半块。随后进行 16 轮迭代，第 i 轮的状态转移为

$$L_i = R_{i-1}, \tag{4}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i), \quad i = 1, 2, \ldots, 16. \tag{5}$$

16 轮结束后，将左右两部分交换次序，再执行逆初始置换 `IP_INV`，得到最终密文：

$$C = IP^{-1}(R_{16} \| L_{16}). \tag{6}$$

(1) 轮函数 F

DES 的安全性核心集中在轮函数 F 中。对任意 $R \in \{0,1\}^{32}$ 和第 i 轮子密钥 $K_i \in \{0,1\}^{48}$，定义

$$F(R, K_i) = P\big(S(E(R) \oplus K_i)\big). \tag{7}$$

其含义如下：

1. **扩展置换 E：**将 32 bit 的右半部分扩展为 48 bit，使其与 48 bit 子密钥维数一致；
2. **异或运算：**扩展结果与子密钥逐位异或，引入密钥影响；
3. **S 盒压缩：**将 48 bit 按 6 bit 一组分为 8 组，经 8 个 S 盒分别映射为 4 bit，共输出 32 bit；
4. **P 置换：**对 S 盒输出执行固定置换，增强扩散效果。

进一步地，若

$$E(R) \oplus K_i = B_1 \| B_2 \| \cdots \| B_8, \quad B_j \in \{0,1\}^6, \tag{8}$$

则 S 盒层输出可以表示为

$$S(E(R) \oplus K_i) = S_1(B_1) \| S_2(B_2) \| \cdots \| S_8(B_8), \tag{9}$$

其中每个 S_j 完成从 6 bit 到 4 bit 的非线性映射。行号由第 1 位与第 6 位组成，列号由中间 4 位组成，这也是 DES 中唯一的非线性环节。

(2) 子密钥生成

DES 需要 16 个 48 bit 子密钥。其生成过程如下：

1. 使用置换选择 `PC1` 从 64 bit 密钥中选出 56 bit 有效位；
2. 将其分为两部分：
$$PC1(K) = C_0 \| D_0, \quad C_0, D_0 \in \{0,1\}^{28}; \tag{10}$$
3. 第 i 轮对 C_{i-1} 和 D_{i-1} 分别循环左移 LS_i 位，得到
$$C_i = \mathrm{LS}_i(C_{i-1}), \quad D_i = \mathrm{LS}_i(D_{i-1}); \tag{11}$$
4. 将 $C_i \| D_i$ 经过 `PC2` 压缩置换为 48 bit 子密钥：
$$K_i = PC2(C_i \| D_i). \tag{12}$$

本实验代码中正是通过 `PC1`、`LS`、`PC2` 三组表和循环左移操作实现上述过程。解密时不必重新设计轮函数，只需按照
$$K_{16}, K_{15}, \ldots, K_1 \tag{13}$$
的逆序使用子密钥即可完成解密，这是 Feistel 结构的重要性质。

(3) 分组、填充与字符串处理

DES 以 8 字节为单位处理数据，而用户输入的聊天消息长度通常不固定，因此需要对消息进行分组和填充。设原始消息字节串长度为 n，则填充值定义为
$$p = 8 - (n \bmod 8), \tag{14}$$
若 $n \bmod 8 = 0$，则补一个完整分组，即 $p = 8$。随后在尾部追加 p 个值均为 p 的字节。填充后的消息长度为
$$n' = n + p, \quad n' \equiv 0 \pmod 8. \tag{15}$$

解密时读取最后一个字节的值作为填充长度，再对尾部进行一致性检查并删除填充。这样既可以处理英文字符，也可以处理 UTF-8 编码下的中文字符，因为程序实际处理的是字节序列而非字符集本身。

(二) TCP 协议与可靠传输机制

TCP（Transmission Control Protocol）是传输层中最常用的面向连接协议，特点是**可靠传输**、**按序到达**、**全双工通信**。对于聊天程序而言，通信双方需要保证消息能够稳定送达、不会乱序，且服务器与客户端都能够在同一条连接上双向发送数据，因此 TCP 比 UDP 更适合作为本实验的通信基础。

(1) 连接建立与可靠传输

TCP 在正式传输数据之前需要先建立连接。客户端主动发起连接请求，服务器端在指定端口监听并接受连接，连接建立成功后，双方才能开始收发消息。TCP 之所以能够提供可靠传输，主要依赖以下机制：

- 使用确认应答机制，发送方能够得知数据是否被对方成功接收；

- 当数据在传输过程中丢失或出错时，可以通过重传机制补发；
- 对接收到的数据按照顺序进行组织，保证应用层读取到的字节顺序正确；
- 通过流量控制和拥塞控制，使传输过程更稳定。

在本实验中，服务端通过 `listen` 和 `accept` 等待连接，客户端通过 `connect` 主动连接服务器。一旦连接建立成功，双方即可在同一条 TCP 连接上持续进行聊天。

(2) TCP 的字节流特性

虽然 TCP 能保证数据可靠到达，但它提供给应用程序的是**字节流**，而不是带边界的消息。也就是说，发送方连续发送两条消息时，接收方并不一定一次 `recv` 就刚好读到其中一条完整消息；也可能一次读到多条消息拼接在一起，或者只读到某条消息的一部分。这就是常说的粘包和拆包问题。

对于普通文本聊天，这个问题有时不明显；但在本实验中，发送的是 DES 加密后的二进制密文，如果接收端不能准确判断每条消息的起止位置，就会导致解密失败或消息错乱。因此，必须在应用层自行补充消息边界信息。

(3) 本实验中的长度头机制

为了解决 TCP 没有天然消息边界的问题，本实验在每条密文前增加 4 字节长度头。发送时，程序先发送密文长度，再发送对应长度的密文字节；接收时，程序先读取固定 4 字节长度头，再根据该长度循环读取完整密文，最后交给 DES 模块解密。

这一机制在实验中的作用主要体现在以下几个方面：

1. 保证接收端能够准确区分每一条应用层消息；
2. 避免由于一次 `recv` 数据不完整而直接解密出错；
3. 使聊天程序能够稳定处理英文、中文以及不同长度的输入内容；
4. 与 `SendAll`、`RecvAll` 循环收发函数配合，保证消息完整传输。

(4) TCP 在本实验中的体现

结合程序实现，TCP 协议在本实验中的体现可以概括为以下流程：

1. 服务器端创建套接字并监听本地端口；
2. 客户端根据 IP 地址和端口主动连接服务器；
3. 连接建立后，双方分别通过发送线程和接收线程完成双向聊天；
4. 每条消息在发送前先由 DES 加密，再以“长度头 + 密文”的形式写入 TCP 连接；
5. 对端读取完整密文后再进行解密并显示明文；
6. 当一端输入 `quit` 或关闭程序时，TCP 连接被释放，另一端能够感知连接断开。

因此，TCP 在本实验中不仅承担了底层数据传输的任务，也直接影响了应用层消息封装方式和程序整体交互逻辑。

(三) Socket 编程与程序工作流程

Socket 可以理解为网络通信端点。Linux 下，TCP 编程的核心并不是直接操作报文，而是通过系统调用完成“建立连接–收发数据–关闭连接”的全过程。

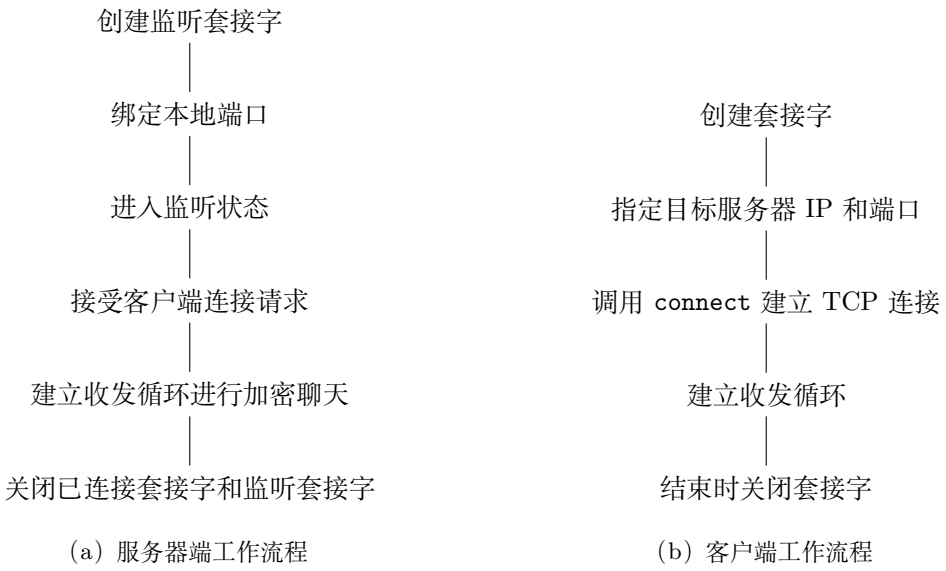
(1) 服务器端与客户端的接口流程

本实验使用的主要 Socket 接口如表 2 所示。

表 2: TCP Socket 主要接口及作用

接口	作用说明
`socket`	创建套接字，返回文件描述符，指定地址族 `AF_INET` 和类型 `SOCK_STREAM`
`bind`	将服务器套接字绑定到本地 IP 和端口
`listen`	将服务器套接字置为监听状态，等待客户端连接
`accept`	从监听队列中取出一个连接请求，生成新的已连接套接字
`connect`	客户端主动向服务器指定地址和端口发起连接
`send`	在已建立连接的套接字上发送字节流
`recv`	在已建立连接的套接字上接收字节流
`shutdown/close`	关闭连接并释放套接字资源

服务器端工作流程为：如图 1 所示，服务器端与客户端在 Socket 编程中的主要执行流程分别如下。



(a) 服务器端工作流程 (b) 客户端工作流程

图 1: Socket 编程中服务器端与客户端的工作流程

(2) 并发收发与线程模型

聊天程序要求用户可以在输入消息的同时接收对方消息。若仅使用单线程阻塞式收发，则程序会在读取终端输入或等待网络数据时阻塞，导致交互不流畅。因此，本实验采用**主线程发送**、

接收线程异步接收的方式：

- 主线程负责读取终端输入、加密消息并发送；
- 接收线程持续调用 `recv` 获取数据并解密输出；
- 通过 `std::atomic<bool>` 变量维护会话运行状态。

这种设计虽然仍属于简单并发模型，但已经能够较好满足一对一聊天实验的需求。

（四） 预共享密钥与本实验的完整通信过程

本实验没有实现密钥协商协议，而是采用**预共享密钥**模型，即通信双方在启动程序之前人工约定同一把 8 字节 DES 密钥，并通过命令行参数 `--key` 传入。这一设计与课程实验要求一致，能够突出“先共享密钥，再进行加密通信”的基本思想。

综合上述原理，本实验单次消息的处理流程如下：

1. 用户在服务端或客户端终端输入一条消息；
2. 程序判断是否为本地控制命令 `quit`，若是则直接关闭连接；
3. 若为普通消息，则先按字节序列进行 DES 分组加密；
4. 发送端将“4 字节长度头 + 密文”写入 TCP 套接字；
5. 接收端先读取长度头，再读取完整密文；
6. 接收端使用同一密钥解密、去填充并恢复原始字符串；
7. 程序将明文显示在终端上，如启用显示密文选项，则同时显示十六进制密文。

因此，本实验从原理上形成了一条完整链路：**用户输入 → DES 加密 → TCP 可靠传输 → DES 解密 → 明文显示**，较好地体现了密码学机制与网络通信机制在实际程序中的结合方式。

四、 实验核心代码解析

1. 命令行参数与程序入口

程序入口位于 main.cpp，负责区分服务端和客户端模式，并解析 --ip、--port、--key、--show-cipher 等参数。若未显式给出是否显示密文，则程序在建立聊天前主动询问用户。

```
if (mode == "server" || mode == "s") {
    config.isServer = true;
} else if (mode == "client" || mode == "c") {
    config.isServer = false;
}

if (arg == "--key" && i + 1 < argc) {
    config.key = argv[++i];
}

if (showCipherOption.has_value()) {
    std::cout << "Show cipher text: "
              << (config.showCipherHex ? "yes" : "no") << '\n';
} else {
    config.showCipherHex = AskShowCipher();
}
```

这一设计的优点在于：

- 运行方式直观，便于实验演示；
- 密钥不写死在业务逻辑中，更符合“双方事先协商密钥”的实验要求；
- 同一个程序即可同时支持服务端和客户端两种角色。

2. DES 子密钥生成

DES 的正确性首先依赖于 16 轮子密钥的生成。程序先将 64 bit 原始密钥转换为位序列，再使用 PC1 选出 56 bit 有效位，随后对左右两半做循环左移，并通过 PC2 压缩得到每轮的 48 bit 子密钥。

```
void DesOperation::GenerateSubKeys() {
    std::uint8_t keyBits[64] = {0};
    ByteToBit(key_.data(), keyBits, 8);

    std::array<std::uint8_t, 28> left{}, right{};
    for (int i = 0; i < 28; ++i) {
        left[i] = keyBits[PC1[i] - 1];
        right[i] = keyBits[PC1[i + 28] - 1];
    }

    for (int round = 0; round < 16; ++round) {
        std::rotate(left.begin(), left.begin() + LS[round], left.end());
        std::rotate(right.begin(), right.begin() + LS[round], right.end());
```

```
        // 经过 PC2 压缩后得到 48 bit 子密钥
    }
}
```

这一部分决定了每轮加密和解密过程中所使用的轮密钥顺序。加密时顺序使用第 1 轮到第 16 轮子密钥，解密时反向使用即可。

3. 消息分组加密与去填充

为了支持任意长度字符串，程序没有只处理单个 64 bit 分组，而是将明文按 8 字节切分。若消息长度不是 8 的整数倍，则按填充值补齐。解密完成后再检查并移除填充字节，恢复原始明文。

```
std::vector<std::uint8_t> DesOperation::EncryptBytes(
    const std::vector<std::uint8_t>& plainText) const {
    std::vector<std::uint8_t> padded = plainText;
    std::uint8_t padding = static_cast<std::uint8_t>(8 - (padded.size() % 8))
        ;
    if (padding == 0) {
        padding = 8;
    }
    padded.insert(padded.end(), padding, padding);

    std::vector<std::uint8_t> cipherText(padded.size(), 0);
    for (std::size_t offset = 0; offset < padded.size(); offset += 8) {
        DESBlock(padded.data() + offset, cipherText.data() + offset, true);
    }
    return cipherText;
}
```

采用这种方式后，程序不仅能够处理英文字符串，也能正确处理 UTF-8 编码的中文字符串。

4. TCP 消息边界处理

本实验的一个重要改进点是：**每条密文前增加 4 字节网络字节序长度头**。发送时先发长度，再发密文；接收时先收长度，再按长度循环接收完整消息。

```
bool Chat::SendPacket(const std::vector<std::uint8_t>& cipherText) {
    const std::uint32_t payloadLength =
        static_cast<std::uint32_t>(cipherText.size());
    const std::uint32_t networkLength = htonl(payloadLength);
    if (!SendAll(reinterpret_cast<const std::uint8_t*>(
            &networkLength), sizeof(networkLength))) {
        return false;
    }
    return SendAll(cipherText.data(), cipherText.size());
}

bool Chat::ReceivePacket(std::vector<std::uint8_t>& cipherText) {
    std::uint32_t networkLength = 0;
```

```
    if (!RecvAll(reinterpret_cast<std::uint8_t*>(
            &networkLength), sizeof(networkLength))) {
        return false;
    }
    const std::uint32_t payloadLength = ntohl(networkLength);
    cipherText.assign(payloadLength, 0);
    return RecvAll(cipherText.data(), cipherText.size());
}
```

与直接调用一次 send/recv 相比，这种设计更符合工程实现要求，也更适合作为拓展实验内容写入报告。

5. 聊天交互与退出控制

在聊天循环中，程序用身份提示符提示当前角色，并在用户输入 quit 时直接关闭本端连接，而不再将其作为普通消息加密发送。这一设计使退出行为更符合用户直觉，也避免了对端收到一条无意义的退出消息。

```
while (isRunning_) {
    std::cout << '[' << selfName << "]> " << std::flush;
    if (!std::getline(std::cin, message)) {
        isRunning_ = false;
        break;
    }

    if (message == kExitCommand) {
        std::cout << selfName << " is closing the connection.\n";
        isRunning_ = false;
        break;
    }

    const auto cipherText = des_.EncryptString(message);
    if (config_.showCipherHex) {
        std::cout << "[cipher hex] " << ToHex(cipherText) << '\n';
    }
    SendPacket(cipherText);
}
```

五、 实验结果和分析

1. 编译与可执行文件结果

程序能够在 Linux 环境下成功构建并生成可执行文件，满足实验对“提交可执行程序”的要求。通过 `ls -l` 和 `file` 命令可验证其具备执行权限，且属于 Linux 下的 ELF 64-bit 可执行文件。

2. 功能测试结果

测试项目	测试内容	结果
DES 算法正确性	使用公开标准测试向量验证，得到标准密文 `85E813540F0AB405`	通过
服务端/客户端连接	服务端监听端口，客户端连接 `127.0.0.1:8888`	通过
英文聊天测试	双方发送英文消息并正确显示	通过
中文聊天测试	双方发送中文消息并正确显示	通过
密文显示测试	启用 `show-cipher` 后显示十六进制密文	通过
消息边界测试	采用长度头后，消息收发稳定，无明显粘包影响	通过
退出控制测试	输入 `quit` 后本端直接断开连接，不再发送退出消息	通过

3. 结果分析

1. **DES 功能正确**。通过标准测试向量验证，可以说明子密钥生成、轮函数和分组变换实现正确。
2. **聊天程序具备基本可用性**。服务端和客户端能够在同一台主机的两个终端中正常建立连接，实现双向收发。
3. **中文支持正常**。程序对输入内容按字节处理，并在接收端恢复为 UTF-8 字符串，因此中文消息也能被正确显示。
4. **拓展实验提升了工程稳定性**。增加长度头后，接收端不再依赖一次 `recv` 就获得完整消息；密文显示选项和角色提示符也使实验演示更加清晰。
5. **当前实现仍有一定局限**。本程序采用单客户端模型，密钥依赖预共享，且底层仍是单 DES，安全性和可扩展性与现代安全通信协议相比仍较弱，但足以满足本次课程实验目标。

4. 程序运行截图

为了更加直观地展示程序运行效果，本文对实验过程中的典型场景进行了截图记录。截图内容主要包括：服务端与客户端的启动连接过程、普通消息与中文消息的双向收发、密文显示功能，以及输入 `quit` 后的连接断开效果。

图 2和图 3 展示了服务器端与客户端启动后的连接过程。可以看到，程序能够正确识别当前运行身份，并在建立连接前询问用户是否显示密文；当客户端连接到服务器后，双方均进入聊天状态，说明 TCP 连接建立正常，程序交互流程符合预期。

图 2: 启动界面

图 3: 连接界面

图 4 展示了双方进行普通消息测试和中文消息测试时的运行结果。从截图中可以观察到，客户端和服务端均能正确接收并显示对方发送的内容，说明程序不仅能够完成英文字符串的收发，也能够正确处理 UTF-8 编码下的中文消息，验证了加密、发送、接收和解密这一完整流程的正确性。同时也展示启用密文显示选项后的运行界面。在该模式下，程序会在终端中额外输出 DES 加密后的十六进制密文。该结果表明，网络中实际传输的是经过加密处理后的密文数据，而不是用户输入的明文，从而体现了本实验先加密、后传输、再解密的基本思想。

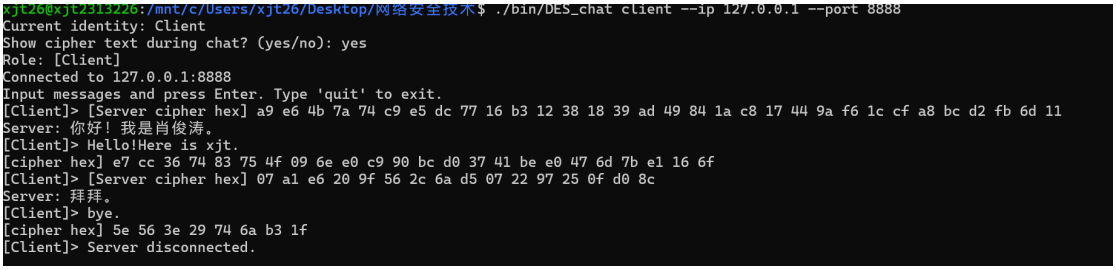
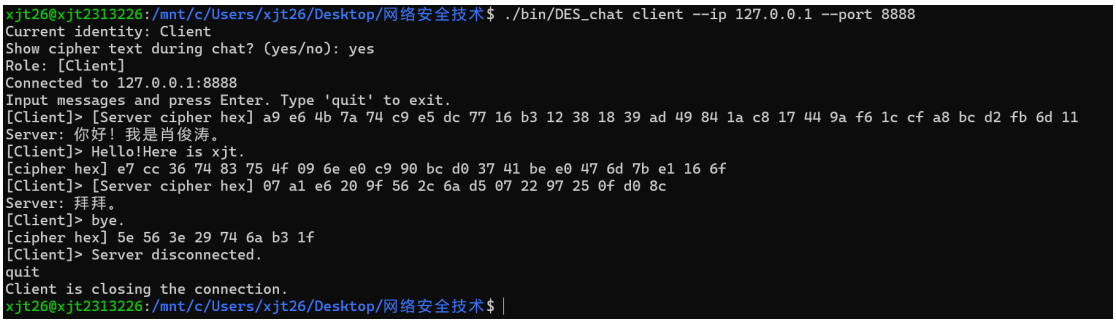


图 4: 聊天界面

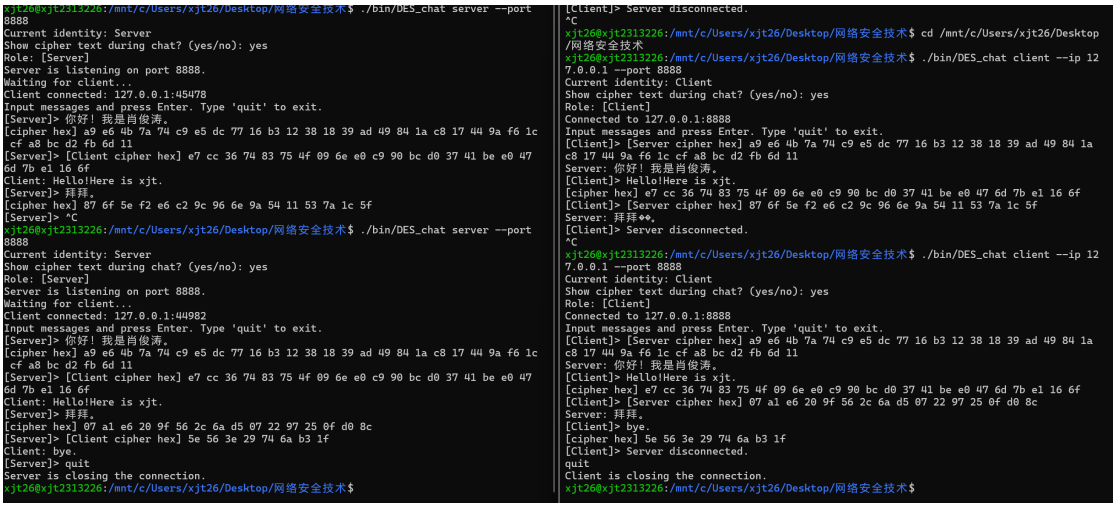
最后图 5 展示了输入 quit 之后的断开连接效果。根据程序设计，quit 被作为本地退出控制命令处理，不再作为普通聊天消息加密发送。截图中可以看到，输入 quit 的一端会直接关闭连接，而另一端能够检测到连接断开并停止继续通信，这说明程序在会话结束控制方面运行正常。



```
xjt26@xjt2313226:/mnt/c/Users/xjt26/Desktop/网络安全技术$ ./bin/DES_chat client --ip 127.0.0.1 --port 8888
Current identity: Client
Show cipher text during chat? (yes/no): yes
Role: [Client]
Connected to 127.0.0.1:8888
Input messages and press Enter. Type 'quit' to exit.
[Client]> [Server cipher hex] a9 e6 4b 7a 74 c9 e5 dc 77 16 b3 12 38 18 39 ad 49 84 1a c8 17 44 9a f6 1c cf a8 bc d2 fb 6d 11
Server: 你好！我是肖俊涛。
[Client]> Hello!Here is xjt.
[cipher hex] e7 cc 36 74 83 75 4f 09 6e e0 c9 90 bc d0 37 41 be e0 47 6d 7b e1 16 6f
[Client]> [Server cipher hex] 07 a1 e6 20 9f 56 2c 6a d5 07 22 97 25 0f d0 8c
Server: 拜拜。
[Client]> bye.
[cipher hex] 5e 56 3e 29 74 6a b3 1f
[Client]> Server disconnected.
quit
Client is closing the connection.
xjt26@xjt2313226:/mnt/c/Users/xjt26/Desktop/网络安全技术$
```

图 5: 退出断连机制

总体来看，实验程序表现良好。



```
xjt26@xjt2313226:/mnt/c/Users/xjt26/Desktop/网络安全技术$ ./bin/DES_chat server --port
8888
Current identity: Server
Show cipher text during chat? (yes/no): yes
Role: [Server]
Server is listening on port 8888.
Waiting for client...
Client connected: 127.0.0.1:45478
Input messages and press Enter. Type 'quit' to exit.
[Server]> 你好！我是肖俊涛。
[cipher hex] a9 e6 4b 7a 74 c9 e5 dc 77 16 b3 12 38 18 39 ad 49 84 1a c8 17 44 9a f6 1c
 cf a8 bc d2 fb 6d 11
[Server]> [Client cipher hex] e7 cc 36 74 83 75 4f 09 6e e0 c9 90 bc d0 37 41 be e0 47
6d 7b e1 16 6f
Client: Hello!Here is xjt.
[Server]> 拜拜。
[cipher hex] 87 6f 5e f2 e6 c2 9c 96 6e 9a 54 11 53 7a 1c 5f
[Server]> ^C
xjt26@xjt2313226:/mnt/c/Users/xjt26/Desktop/网络安全技术$ ./bin/DES_chat server --port
8888
Current identity: Server
Show cipher text during chat? (yes/no): yes
Role: [Server]
Server is listening on port 8888.
Waiting for client...
Client connected: 127.0.0.1:44982
Input messages and press Enter. Type 'quit' to exit.
[Server]> 你好！我是肖俊涛。
[cipher hex] a9 e6 4b 7a 74 c9 e5 dc 77 16 b3 12 38 18 39 ad 49 84 1a c8 17 44 9a f6 1c
 cf a8 bc d2 fb 6d 11
[Server]> [Client cipher hex] e7 cc 36 74 83 75 4f 09 6e e0 c9 90 bc d0 37 41 be e0 47
6d 7b e1 16 6f
Client: Hello!Here is xjt.
[Server]> 拜拜。
[cipher hex] 07 a1 e6 20 9f 56 2c 6a d5 07 22 97 25 0f d0 8c
[Server]> [Client cipher hex] 5e 56 3e 29 74 6a b3 1f
Client: bye.
[Server]> quit
Server is closing the connection.
xjt26@xjt2313226:/mnt/c/Users/xjt26/Desktop/网络安全技术$
```

```
[Client]> Server disconnected.
^C
xjt26@xjt2313226:/mnt/c/Users/xjt26/Desktop/网络安全技术$ cd /mnt/c/Users/xjt26/Desktop
/网络安全技术
xjt26@xjt2313226:/mnt/c/Users/xjt26/Desktop/网络安全技术$ ./bin/DES_chat client --ip 12
7.0.0.1 --port 8888
Current identity: Client
Show cipher text during chat? (yes/no): yes
Role: [Client]
Connected to 127.0.0.1:8888
Input messages and press Enter. Type 'quit' to exit.
[Client]> [Server cipher hex] a9 e6 4b 7a 74 c9 e5 dc 77 16 b3 12 38 18 39 ad 49 84 1a
c8 17 44 9a f6 1c cf a8 bc d2 fb 6d 11
Server: 你好！我是肖俊涛。
[Client]> Hello!Here is xjt.
[cipher hex] e7 cc 36 74 83 75 4f 09 6e e0 c9 90 bc d0 37 41 be e0 47 6d 7b e1 16 6f
[Client]> [Server cipher hex] 87 6f 5e f2 e6 c2 9c 96 6e 9a 54 11 53 7a 1c 5f
Server: 拜拜◆◆。
[Client]> Server disconnected.
^C
xjt26@xjt2313226:/mnt/c/Users/xjt26/Desktop/网络安全技术$ ./bin/DES_chat client --ip 12
7.0.0.1 --port 8888
Current identity: Client
Show cipher text during chat? (yes/no): yes
Role: [Client]
Connected to 127.0.0.1:8888
Input messages and press Enter. Type 'quit' to exit.
[Client]> [Server cipher hex] a9 e6 4b 7a 74 c9 e5 dc 77 16 b3 12 38 18 39 ad 49 84 1a
c8 17 44 9a f6 1c cf a8 bc d2 fb 6d 11
Server: 你好！我是肖俊涛。
[Client]> Hello!Here is xjt.
[cipher hex] e7 cc 36 74 83 75 4f 09 6e e0 c9 90 bc d0 37 41 be e0 47 6d 7b e1 16 6f
[Client]> [Server cipher hex] 07 a1 e6 20 9f 56 2c 6a d5 07 22 97 25 0f d0 8c
Server: 拜拜。
[Client]> bye.
[cipher hex] 5e 56 3e 29 74 6a b3 1f
[Client]> Server disconnected.
quit
Client is closing the connection.
xjt26@xjt2313226:/mnt/c/Users/xjt26/Desktop/网络安全技术$
```

图 6: 总体视图

六、 收获与反思

通过本次实验，我对加密算法实现与网络程序设计之间的联系有了更加直观的理解。之前对 DES 的认识更多停留在理论层面，而在实际编码过程中，需要将置换、分组、轮函数、填充和解密流程严格落到字节与位级别，这使我对分组加密的整体流程理解得更加深入。

同时，本实验也让我进一步掌握了 Linux Socket 编程的基本过程，包括服务器监听、客户端连接、阻塞收发以及线程化接收等内容。特别是在将 DES 模块与 TCP 聊天模块集成之后，我更加清楚地认识到两者结合的作用。

后续若继续完善该项目，还可以考虑加入更安全的加密算法、密钥协商机制以及多客户端支持，以进一步贴近真实网络安全应用场景。